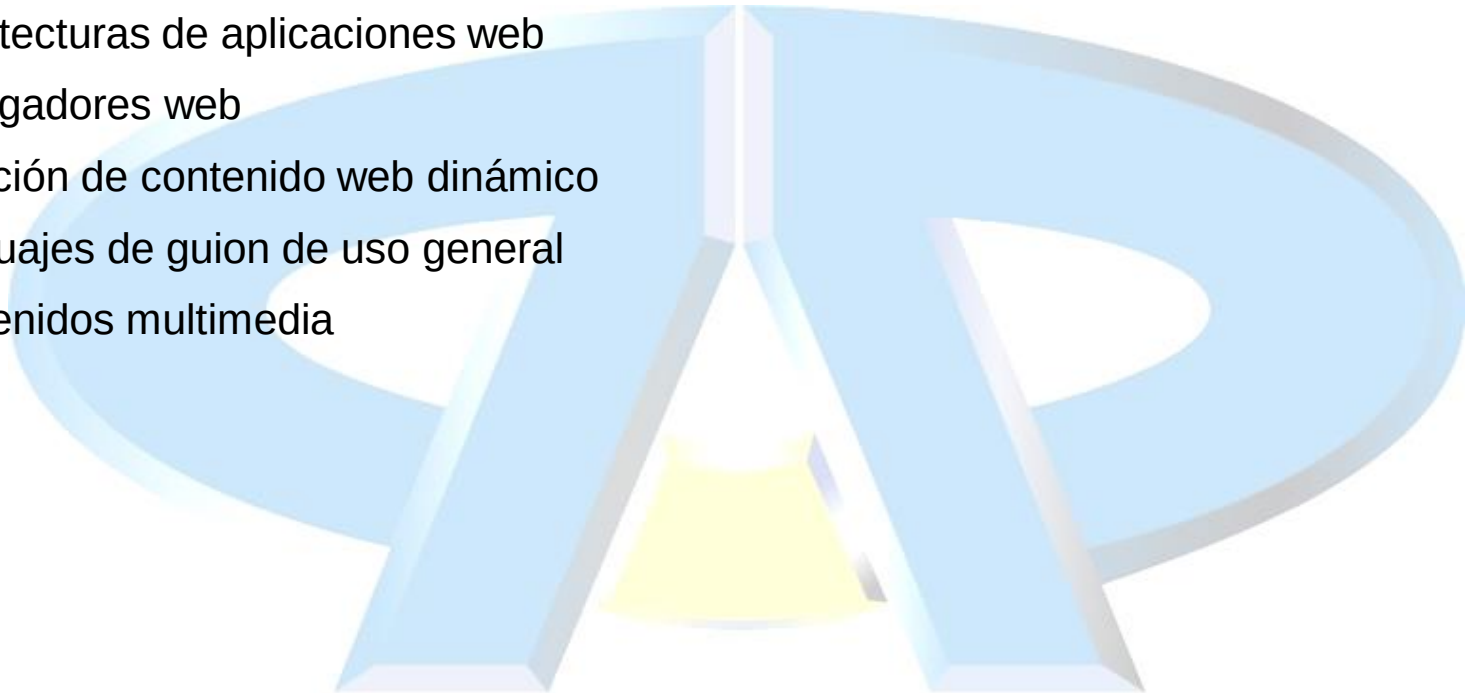


Desarrollo y reutilización de componentes software y multimedia mediante lenguajes de guion

MF0491 – UF1842

UF1842: DESARROLLO Y REUTILIZACIÓN DE COMPONENTES SOFTWARE Y MULTIMEDIA MEDIANTE LENGUAJES DE GUION (90 horas)

- Arquitecturas de aplicaciones web
- Navegadores web
- Creación de contenido web dinámico
- Lenguajes de guion de uso general
- Contenidos multimedia





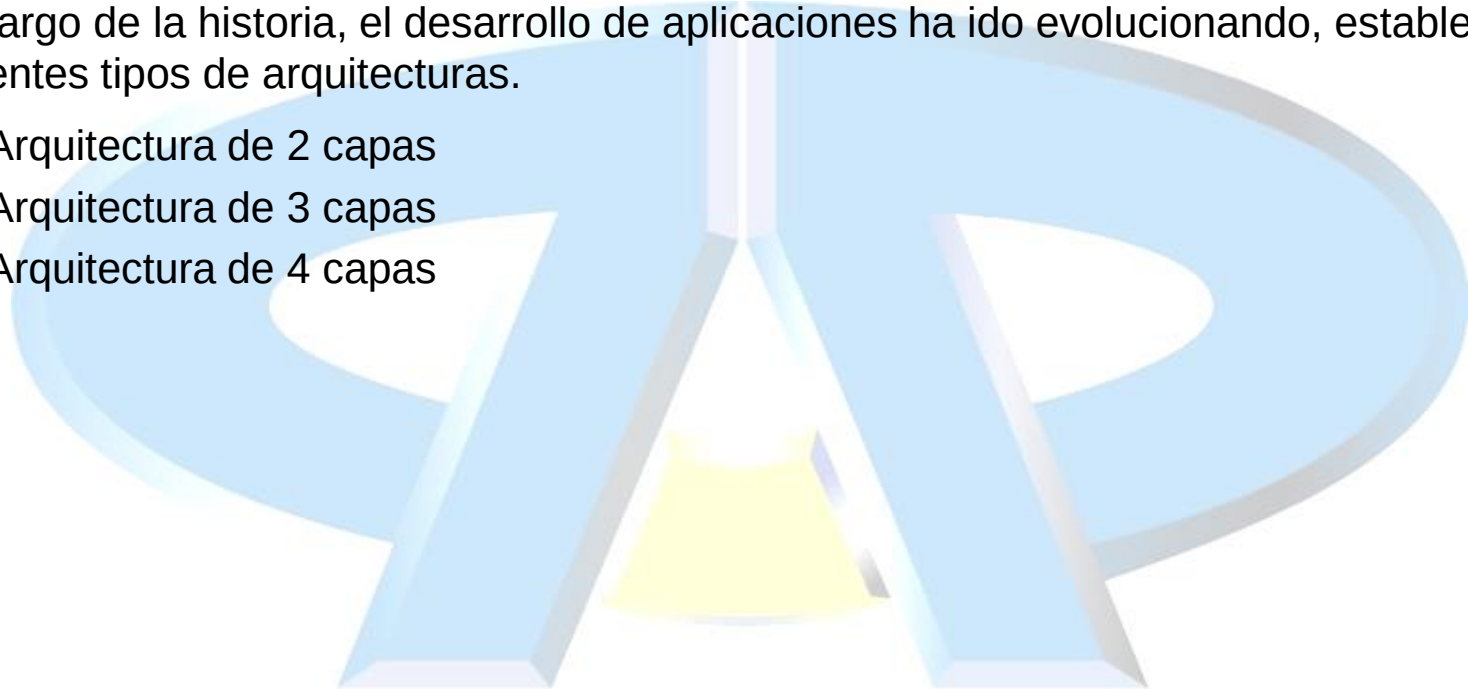
ARQUITECTURA DE APLICACIONES WEB

ESQUEMA GENERAL

- TCP/IP implementa un mecanismo fiable para la transmisión de datos entre ordenadores.
- TCP/IP permite que un programador pueda establecer comunicación de datos entre dos programas de aplicación, tanto si ambos se están ejecutando en la misma maquina, como en maquinas distintas unidas por algún camino físico
- TCP/IP especifica los detalles y mecanismos para la transmisión de datos entre dos aplicaciones comunicantes, pero no dictamina cuando ni porque deben interactuar ambas aplicaciones, ni siquiera especifica como debería estar organizada una aplicación que se va a ejecutar en un entorno distribuido.
- En la práctica el esquema de programación mas utilizado para la implementación de aplicaciones distribuidas es el paradigma cliente-servidor.
 - La motivación fundamental para el empleo del paradigma cliente-servidor surge del problema del rendezvous (encuentro)
- Según este modelo, en cualquier par de aplicaciones comunicantes, una de las partes implicadas en la comunicación (**el servidor**) debe iniciar su ejecución y esperar indefinidamente hasta que la otra parte (**el cliente**) contacte con ella.

ARQUITECTURA EN CAPAS

- Definiremos arquitectura como una forma de estructurar los elementos de un software.
- A lo largo de la historia, el desarrollo de aplicaciones ha ido evolucionando, estableciéndose diferentes tipos de arquitecturas.
 - Arquitectura de 2 capas
 - Arquitectura de 3 capas
 - Arquitectura de 4 capas



ARQUITECTURA EN CAPAS

- Arquitectura en 2 capas
 - Capa de aplicación:
 - Normalmente reside en el cliente, se encarga de todas las tareas de presentación (interfaz) y de aplicación
 - Capa de datos:
 - También llamada repositorio, reside en el servidor y se encarga de garantizar el almacenamiento y la persistencia de los datos.
- Inconvenientes:
 - La capa de aplicación se recarga, entremezclando aspectos típicos del manejo de la interfaz con las reglas del negocio
 - El nivel de aplicación puede ser demasiado pesado para el cliente, lo que genera un bajo rendimiento, aunque actualmente, con las capacidades de los equipos, no suele representar un problema

ARQUITECTURA EN CAPAS

- Arquitectura en 3 capas
 - Capa de presentación:
 - Reside en el cliente y se encarga de presentar los resultados al cliente y de capturar la información introducida por este.
 - Capa de aplicación:
 - Puede residir en el cliente o en el servidor, aunque es preferible que sea en este último y se encarga de la implementación de la lógica de negocio (las funcionalidades de la aplicación) y de la interacción con la capa de presentación.
 - Capa de datos:
 - También llamada repositorio, reside en el servidor y se encarga de garantizar el almacenamiento y la persistencia de los datos.

ARQUITECTURA EN CAPAS

- Arquitectura en 4 capas
 - Capa de presentación:
 - Reside en el cliente y se encarga de presentar los resultados al cliente y de capturar la información introducida por este.
 - Capa de controlador:
 - Se encarga de la comunicación con la capa de presentación, gestionando el envío de los datos, su formateo, internacionalización,...
 - Capa del modelo:
 - Puede residir en el cliente o en el servidor, aunque es preferible que sea en este último y se encarga de la implementación de la lógica de negocio (las funcionalidades de la aplicación)
 - Capa de datos:
 - También llamada repositorio, reside en el servidor y se encarga de garantizar el almacenamiento y la persistencia de los datos.

INTERACCIÓN ENTRE CAPAS

- Arquitectura top-down de capas
 - Los elementos de una capa $i+1$ pueden enviar solicitudes de servicio a elementos de la capa inferior i .
 - Típicamente se produce una cascada de solicitudes, es decir para satisfacer una solicitud a una capa $i+2$, ésta requiere enviar varias solicitudes a la capa $i+1$; cada una de estas solicitudes a la capa $i+1$ genera a su vez un conjunto de solicitudes a la capa i y así sucesivamente.
 - Una arquitectura top-down es laxa (o no estricta) si los elementos de una capa $i+1$ pueden enviar solicitudes de servicio directamente a un elemento de cualquiera de las i capas inferiores
- Arquitectura bottom-up de capas
 - Cada elemento de una capa i puede notificar a elementos de la capa superior $i+1$ de que ha ocurrido algún evento de interés.
 - La capa $i+1$ puede juntar varios eventos antes de notificar a su vez al elemento de la capa $i+2$.
 - Una arquitectura bottom-up también puede ser no estricta si el elemento de la capa i puede notificar a cualquier elemento de cualquier capa superior a la capa i .
- Arquitectura bidireccional de capas
 - En su forma más común involucra dos pilas de N capas que se comunican entre si. El ejemplo más conocido es el de los protocolos en Redes de Computadores.



CREACIÓN DE CONTENIDO WEB DINÁMICO – LENGUAJES DE GUIÓN DE USO GENERAL

JAVASCRIPT

- JavaScript es una de las múltiples aplicaciones que surgieron para extender las capacidades del lenguaje HTML.
- JavaScript es un lenguaje script orientado a documento, como pueden ser los lenguajes de macros que tienen muchos procesadores de texto.
- Características:
 - Todo el código (sentencias) está dentro de funciones.
 - Puede existir código fuera de las funciones, para definir variables globales o ejecutarse con la carga de la página.
 - Las funciones se desarrollan entre las etiquetas **<script>** y **</script>**
 - La llamada a la función se hace a través de un evento de un elemento del documento.
 - Toda sentencia acaba con ;
 - Los bloques van entre { y }

CARGA DE FICHEROS

- Las etiquetas **<script>** se colocan entre las etiquetas **<head>** y **</head>**.
 - También se puede escribir el código en un fichero externo y enlazarlo mediante el atributo **src** de la etiqueta **script**
 - El fichero llevará la extensión **.js**
- La descarga de los ficheros se produce según se van *encontrando*.
- Si el script necesita acceder al contenido del **html** sin interacción del usuario:
 - Se puede colocar el elemento **script** al final del **body**
 - Si es un fichero externo, se puede marcar con el atributo **defer**
 - Retrasa la ejecución hasta que finalice la carga del html
 - También se puede marcar con el atributo **async** para permitir la carga en paralelo con el html
 - No se puede determinar el orden de carga de los ficheros

FUNCIONES

- Las funciones son un conjunto de sentencias (bloque de código) que le especifican al programa las operaciones que debe realizar.
 - Son útiles para evitar la repetición de líneas y modular el código.
- Se les pasa como aquellos elementos que necesitan para realizar su trabajo

```
function nombre_función(var1, var2, ....., varN) {  
    sentencia(s);  
}
```

- La llamada a una función se realiza desde un evento de una etiqueta de HTML
<elemento evento="nombre_función(var1, var2, ..., varN);" >

FUNCIONES

- Una función también puede devolver un valor como resultado de su ejecución
 - Ese valor, del tipo que sea, se devolverá mediante la sentencia **return**

```
function menor (numero1, numero2) {  
    var numMenor;  
    if (numero1 < numero2) {  
        numMenor = numero1;  
    } else {  
        numMenor = numero2;  
    }  
    return numMenor;  
}
```

FUNCIONES

- En la versión de 2015 se agregaron las funciones flecha
- Permiten una forma abreviada de escribir funciones
 - Anteriormente:
let funcion1 = function(a, b) { return a+b; }
 - Ahora:
let funcion1 = (a, b) => a+b;
- Los paréntesis de los parámetros se pueden omitir si solamente hay uno
 - Si no hay ningún parámetro, son obligatorios
- Las llaves del cuerpo de la función se pueden omitir si solamente hay una instrucción. En otro caso son obligatorios.

EVENTOS

- Permite capturar la ocurrencia de determinados eventos que ocurren en el documento web o en los elementos, realizando acciones personalizadas cuando ocurren.

EVENTO	SE PRODUCE AL...
onLoad	Acabar de cargar una página o frame (entrar).
onUnload	Descargar una página o frame (salir).
onMouseOver	Pasar el ratón por encima de un elemento
onMouseOut	Dejar de estar el ratón encima de un elemento
onMouseEnter	Al entrar el ratón en un elemento
onMouseLeave	Al salir el ratón de un elemento
onMouseMove	Mover el ratón por el documento.
onWheel	Al mover la rueda del ratón
onKeyDown	Al pulsar una tecla, cuando baja
onKeyUp	Al soltar la tecla, después de pulsarla
onClick	Hacer clic con el botón principal del ratón.
onDbClick	Hacer doble clic con el ratón.
onContextMenu	Hacer clic con el botón secundario del ratón.
onChange	Modificar texto o un select. Sucede al perder el foco.
onInput	Al producirse una entrada del usuario.
onSelect	Seleccionar texto en un control de edición.
onFocus	Situar el foco en un control.
onBlur	Perder el foco un control.
onSubmit	Enviar un formulario.
onReset	Reiniciar un formulario.

VARIABLES

- Espacio de memoria con un nombre reservado para guardar información mientras la página está cargada.
- El primero paso para poder trabajar con variables es su declaración, donde se les da un nombre y su ámbito.
- Para dar nombre a una variable hay que tener en cuenta las siguientes normas:
 - Pueden contener letras, números, guión bajo o dolar
 - No pueden contener espacios.
 - Distingue entre mayúsculas y minúsculas.
 - No pueden contener acentos, puntos o cualquier signo gramatical.
 - No pueden comenzar con un dígito ni contener la letra “ñ”.
 - Nombre único y exclusivo para cada variable salvo que estén en dos funciones distintas.
 - No pueden usarse palabras reservadas por JavaScript

var nombre_variable;

var nombre_variable=valor;

VARIABLES

- El ámbito de una variable define si la variable se puede utilizar en cualquier parte del documento (es global), o si sólo se podrá utilizar dentro de una función determinada (es local).
 - La declaración de las variables globales se realiza dentro de las etiquetas **<script>** pero fuera de cualquier función.
 - La declaración de las variables locales se realiza dentro de la función en que se quiera usar esa variable.
- La declaración del tipo de datos que contendrá la variable es realizada automáticamente por JavaScript y depende del primer valor que se guarde en la variable.
- La conversión de datos en JavaScript es automática.
- Transforma los datos de todas las variables en una expresión según el tipo de la primera variable que aparece en la expresión.
 - No es muy segura y puede acarrear muchos problemas, así que es recomendable realizar la conversión de manera explícita mediante las siguientes funciones:

De texto a número entero.	<code>var_numerica = parseInt (var_texto);</code>
De texto a coma flotante (decimal).	<code>var_numerica=parseFloat(var_texto);</code>
De numérica a texto.	Es automática y sin peligro.

VARIABLES

- A partir del año 2015, se añadió el ámbito de bloque en las variables de JavaScript
 - Se puede declarar una variable con la palabra **let**
 - La variable solamente podrá ser accedida en el bloque en el que se declara { ... }
 - Las variables declaradas con **let** no se pueden volver a declarar en el mismo bloque
 - Las que se declaran con **var** sí
 - También se pueden declarar variables que no pueden modificar su valor mediante la palabra **const**
 - Es obligatorio darle valor en el momento de su declaración
 - Los arrays u objetos declarados con **const** pueden modificar su contenido
 - Las variables declaradas con **let** o **const** no se pueden utilizar antes de su declaración
 - Las que se declaran con **var** sí
 - JavaScript soporta un modo de trabajo llamando strict mode. En él todas las variables tienen que ser declaradas antes de ser utilizadas.
 - Se incluye “**use strict**”; al comienzo del script o función.

LITERALES

- Al inicializar una variable se le asigna un valor invariable, una expresión literal constante.
- Los tipos de literales son los mismos que en las variables, según el primer dato que se almacene será de un tipo u otro.

- Los números representan literales numéricos enteros

1 2 3

- Los números con decimales representan números en coma flotante

1.2 3.1416 123.56

- Los números también se pueden representar en notación científica (e representa 10 elevado a)

123e4

- **NaN**, **Infinity** y **-Infinity** también son números
- Los caracteres entre comillas (dobles o simples) representan Strings (cadenas de texto)

“Javascript” “HTML” ‘CSS’

- Los valores lógicos se representan con dos palabras

true false

STRING LITERALS

- A partir de 2015, se añadió una nueva funcionalidad en JavaScript
- Los string literals o template literals utilizar acentos (`) en vez de comillas (“ o ‘) para definir los strings
- La diferencia con los strings tradicionales es que permite incluir variables dentro de ellos sin necesidad de utilizar la concatenación ni realizar conversiones de tipos
 - También soporta la inclusión de operaciones, añadiendo el resultado
- Las variables o expresiones se escribirán con `${ ... }`

OPERADORES

- JavaScript define tres tipos de operadores

- Aritméticos

+	Suma - Concatenación	%	Resto de la división
-	Resta.	**	Potencia
*	Multiplicación	--	Decremento en 1
/	División	++	Incremento en 1

- Relacionales

<	Menor que.	==	Igual
>	Mayor que.	===	Igual valor y tipo
<=	Menor o igual.	!=	Distinto
>=	Mayor o igual	!==	Distinto valor o tipo

- Lógicos

&&	Y(AND)
	O(OR)
!	NO (NOT)

- También existe un operador para la asignación:

variable=valor;

- La prioridad entre los tipos de operadores coincide con el orden de esta diapositiva

Tablas de verdad de las operaciones lógicas

A	B	A && B	A B
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

OPERADORES

- Los operadores de asignación también se pueden escribir de forma abreviada:
 - += → variable = variable + valor
 - -= → variable = variable - valor
 - *= → variable = variable * valor
 - /= → variable = variable / valor
 - %= → variable = variable % valor
 - **= → variable = variable ** valor
- Existen operadores de tipo
 - **typeof**: Devuelve el tipo de una variable
 - **instanceof**: Devuelve verdadero si un objeto es una instancia de una clase
- También hay un operador ternario
 - comparación ? valor1 : valor2
 - Si el resultado de la comparación es verdadero, devuelve el valor1, si es falso, devuelve el valor2

COMENTARIOS

- Los comentarios en Javascript pueden ser de dos tipos:
 - De línea:
 - Se escriben dos barras de división al principio de la línea de comentario
// Línea a comentar
 - De bloque
 - Se encierra el bloque entre /* y */
/* Bloque de varias líneas a comentar */

SENTENCIAS DE CONTROL

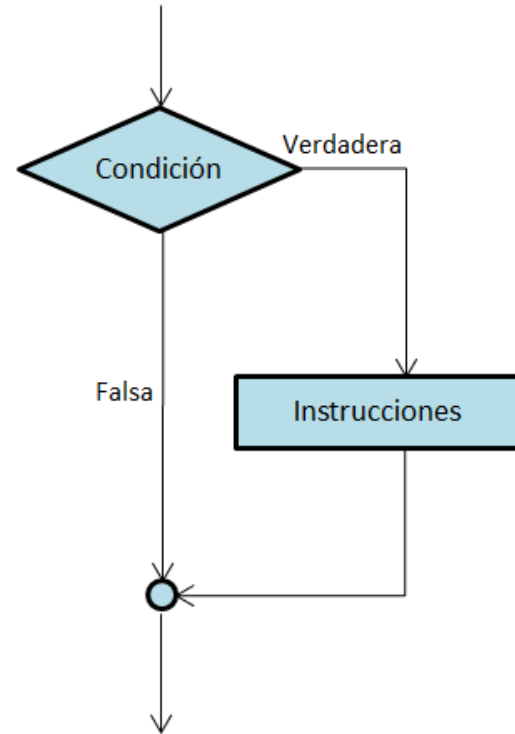
- Las instrucciones de un programa se procesan en orden secuencial actuando como separador el ;
- Pero en determinadas circunstancias es necesario romper ese orden, lo que se puede realizar de dos formas:
 - Mediante sentencias condicionales, en las que la evaluación de una condición provocará la bifurcación del orden de ejecución
 - Mediante sentencias repetitivas, que provocan la repetición de un conjunto de instrucciones hasta que se cumpla una condición o durante un determinado número de veces

SENTENCIAS DE CONTROL

- Condicional simple

```
if (condición) {  
    sentencia(s);  
}
```

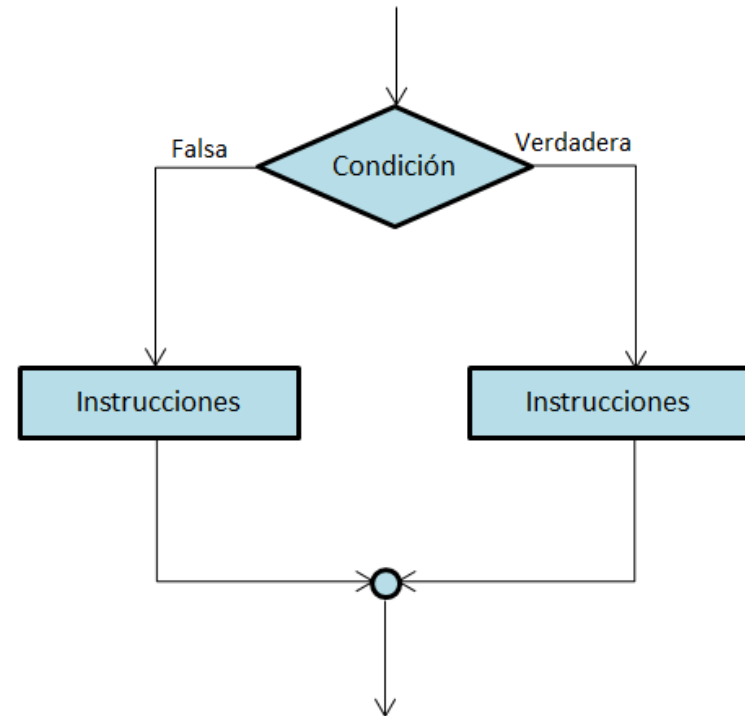
- Se comprueba la condición (lógica) y si se evalúa a verdadero, entonces se ejecuta el código que va en el bloque



SENTENCIAS DE CONTROL

- Condicional doble

```
if (condición) {  
    sentencia(s);  
} else {  
    sentencias(s);  
}
```

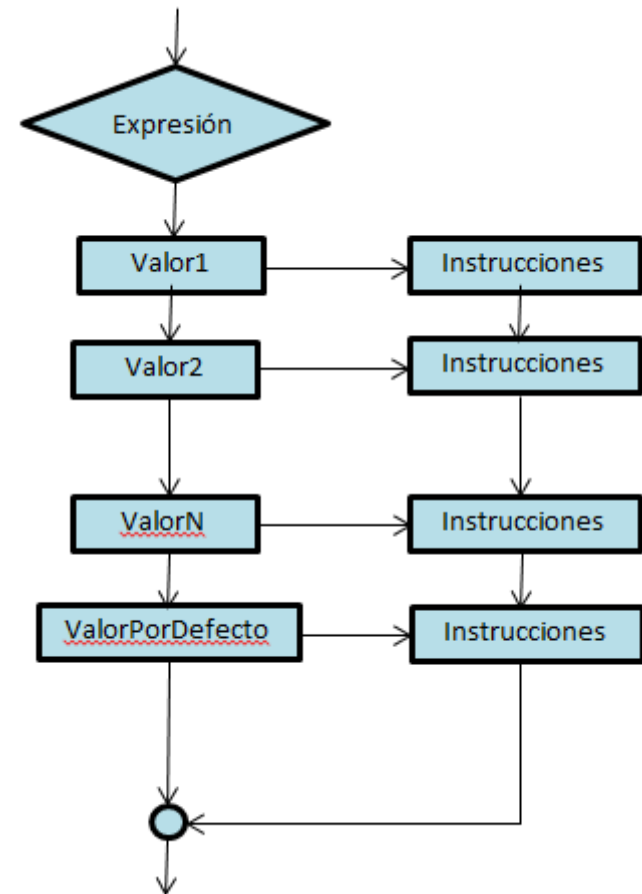


- Se comprueba la condición
 - Si se evalúa a verdadero, entonces se ejecuta el código que va en el bloque correspondiente
 - Si se evalúa a falso, entonces se ejecuta el código del otro bloque

SENTENCIAS DE CONTROL

- Condicional múltiple

```
switch (expresión) {  
    case valor1:  
        sentencia(s);  
        break;  
    case valor2:  
        sentencia(s);  
        break;  
    case valorN:  
        sentencia(s);  
        break;  
    default:  
        sentencia(s);  
}
```



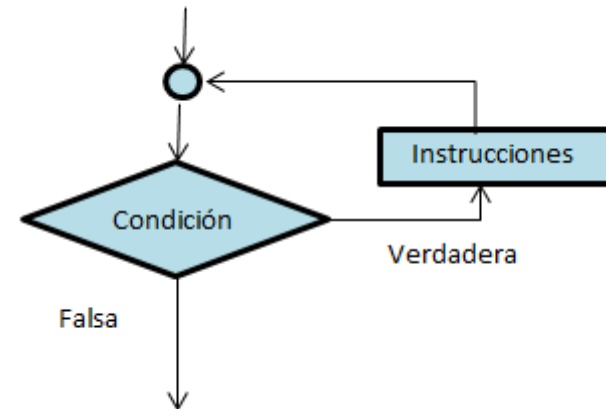
SENTENCIAS DE CONTROL

- Condicional múltiple
 - Se comprueba el valor de la expresión y se compara con los distintos valores que contienen cada una de las sentencias **case**
 - La primera que coincida, se ejecutan las instrucciones asociadas a ese **case**
 - Una vez ejecutadas, se continuará con el resto de instrucciones, a menos que se incluya una instrucción **break**, que provoca la expulsión del bloque
 - En caso de que ninguna de las sentencias **case** coincida, si existe una sentencia **default**, se ejecutarán las sentencias asociadas a esta
 - La sentencia **default** no tiene por qué estar colocada en última posición, pero sí será la última en ser evaluada.
 - La expresión incluida en la sentencia **switch** puede ser de cualquier tipo
 - La comparación del switch es estricta (===)
 - Los valores incluidos en las sentencias **case** pueden ser constantes, variables o expresiones

SENTENCIAS DE CONTROL

- Bucle “mientras”

```
while (condición) {  
    sentencia(s);  
}
```

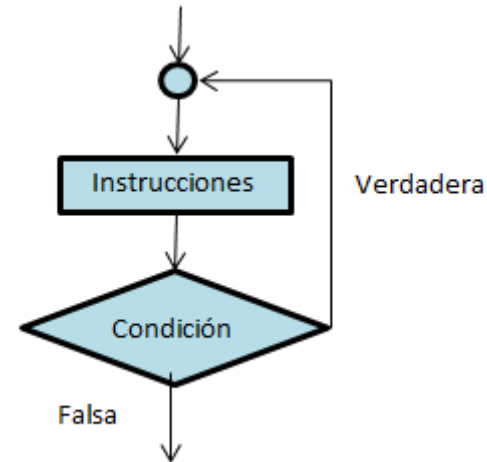


- Primero se evalúa la condición
 - Si es verdadera se ejecutan las sentencias asociadas y se vuelve a evaluar la condición
 - Si es falsa se finaliza el bucle y se continúa

SENTENCIAS DE CONTROL

- Bucle “hacer mientras”

```
do {  
    sentencia(s);  
} while (condición);
```

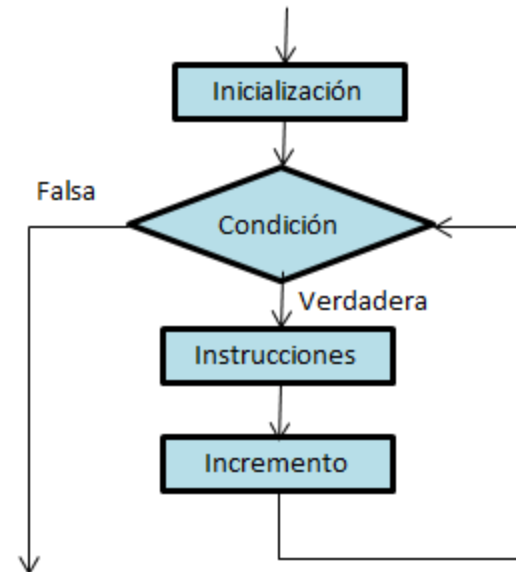


- Primero ejecutan las instrucciones
- A continuación se evalúa la condición
 - Si es verdadera se vuelven a ejecutar las instrucciones y se vuelve a evaluar la condición
 - Si es falsa se finaliza el bucle y se continúa

SENTENCIAS DE CONTROL

- Bucle “para”

```
for (inicialización; condición; incremento) {  
    sentencia(s);  
}
```



- Cuando comienza el bucle, se inicializa la variable contador
- A continuación se comprueba la condición
 - Si es verdadera se ejecutan las sentencias, se incrementa el contador y se vuelve a evaluar la condición
 - Si es falsa se finaliza el bucle y se continúa

SENTENCIAS DE CONTROL

- Bucle “para-de”

```
for (variable of colección) {  
    sentencia(s);  
}
```

- Recorre una colección de elementos, almacenando cada uno de ellos en la variable, pudiéndose usar esta en las sentencias de dentro del bucle
 - El recorrido es de uno en uno y siempre hacia adelante
- La colección puede ser un array o un string

SENTENCIAS DE CONTROL

- Bucle “para-en”

```
for (variable in objeto) {  
    sentencia(s);  
}
```

- Recorre todas las propiedades de un objeto
- Hay que tener en cuenta que tal vez no se sepa de antemano el orden de las propiedades

ARRAYS

- Una array es una colección de valores que (en Javascript) no tienen por qué ser del mismo tipo, almacenados bajo un nombre común en posiciones consecutivas de memoria.
- A un elemento específico de un array se accede mediante su índice, siendo el primer índice el 0.

```
let nombreArray = new Array(tamaño);  
nombreArray[indice] = valor;
```

- Es posible inicializar los arrays a una lista de valores, indicándola en el momento de construir el array.

```
let nombreArray = new Array(elemento1, elemento2,..., elementoN);  
let nombreArray = [elemento1, elemento2,..., elementoN];
```

- El tamaño del array se obtiene con la propiedad **length**
nombreArray.length

CLASES

- En el ECMAScript 2015 se añadió la definición de clase, mediante la palabra reservada **class**
- También se declaran los métodos dentro de la clase

```
class Rectangulo {  
    constructor(alto, ancho) {  
        this.alto = alto;  
        this.ancho = ancho;  
    }  
    area() {  
        return this.alto * this.ancho;  
    }  
}
```

- La palabra reservada **constructor** representa el método utilizado para crear un objeto de esa clase

OBJETOS

- Los objetos son instancias de las clases, es decir, cuando se reserva espacio en memoria para almacenar una clase y sus atributos toman valores
- Las clases se instancian llamando al constructor de la clase

let objetoRectangulo = new Rectangulo(10, 10)

- En Javascript se pueden crear objetos sin necesidad de definir las clases
 - Se escribe el objeto entre llaves, separando con comas cada una de las parejas nombre del atributo : valor del atributo

let objetoRectangulo = { alto: 10, ancho: 10 }

THIS

- El elemento `this` representa el “yo” para los objetos
 - Se utiliza para acceder a las propiedades y métodos cuando estamos definiendo un objeto
 - También se puede utilizar para hacer referencia al elemento sobre el que recae la acción que desencadena el evento

```
function ocultar(elemento) {  
    elemento.style.visibility = “hidden”;  
}
```

```
<img src=“imagen.jpg” onclick=“ocultar(this);”/>
```

FECHAS

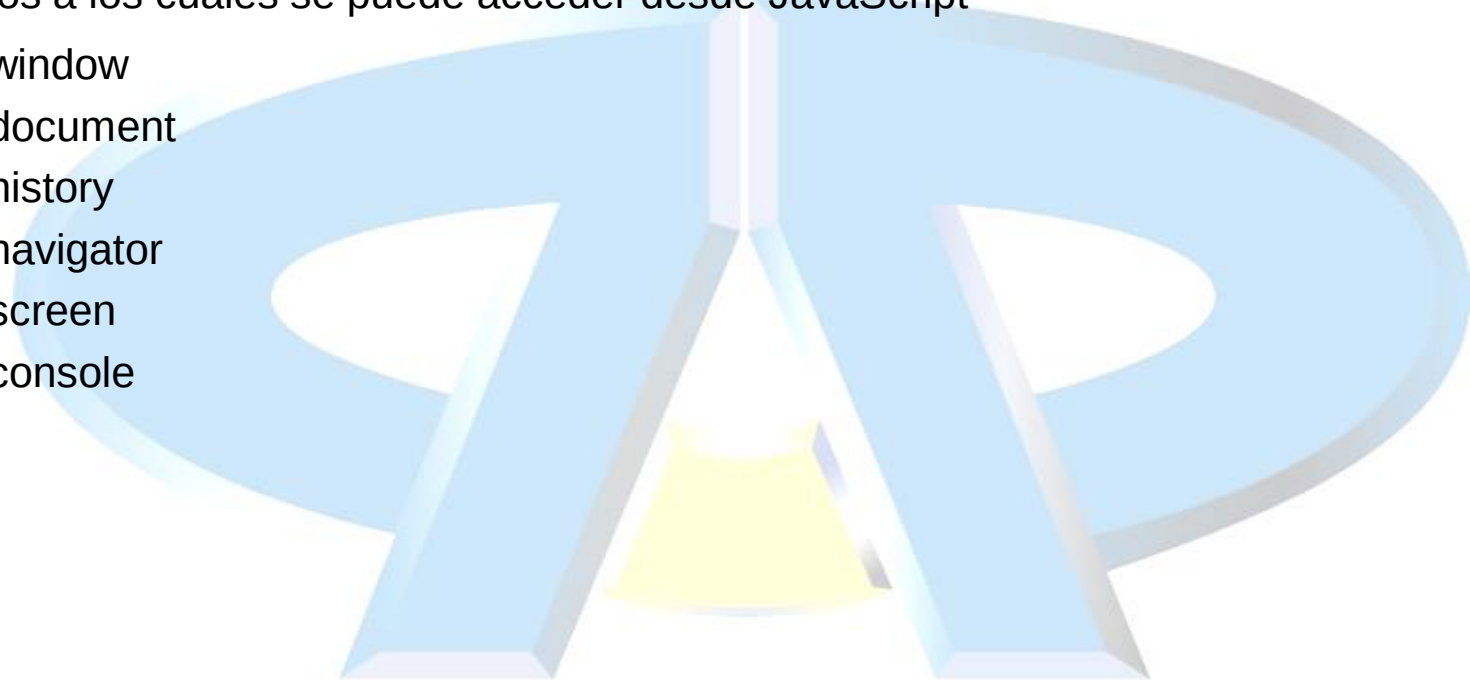
- Para la manipulación de fechas en JavaScript se utiliza el objeto Date.
- Existen cuatro formas de crear una fecha:
 - **var fecha = new Date()**
 - Crea una fecha con la fecha y hora actual
 - **var fecha = new Date(año, mes, día, hora, minuto, segundo, milisegundos)**
 - Crea una fecha con los valores de los parámetros indicados
 - Si se pasan menos parámetros, se van eliminando por el final, hasta un mínimo de 2 parámetros
 - Los meses van de 0 a 11
 - **var fecha = new Date(milisegundos)**
 - Crea una fecha calculándola a partir del número de milisegundos que pasan desde el 1 de enero de 1970
 - **var fecha = new Date(fechaString)**
 - Crea una fecha convirtiendo la cadena de texto pasada
 - aaaa-mm-dd
 - mm/dd/aaaa

MODIFICACIÓN DEL HTML (DOM)

- El DOM de HTML es un modelo de objetos estándar y una interfaz de programación para HTML. Define:
 - Los elementos
 - Las propiedades
 - Los métodos
 - Los eventos
- Es posible añadir o eliminar elementos del DOM, así como modificar sus propiedades
- Se puede hacer mediante métodos de los elementos
 - **createElement(nombreElemento), createTextNode(texto), appendChild(elemento), insertBefore(elementoPadre, elementoNuevo), remove(),...**
- Modificando el HTML
 - Asignándole valor a la propiedad **innerHTML**
 - Con el método **insertAdjacentHTML(posicion, html)**, donde posición puede valer:
 - 'beforebegin', 'beforeend', 'afterbegin', 'afterend'

OBJETOS PREDEFINIDOS (BOM)

- Por defecto, cuando se visualiza una página web en un navegador, existen una serie de objetos a los cuales se puede acceder desde JavaScript
 - window
 - document
 - history
 - navigator
 - screen
 - console



OBJETO WINDOW

- Permite acceder a las propiedades y métodos de la ventana en la que se está visualizando la página web

METODO	DESCRIPCIÓN	SINTAXIS
open	Abrir ventanas.	var=window.open("url","name","atrbs");
close	Cerrar ventanas.	var.close();
print	Imprime el documento.	var.print();
alert	Abre ventanas alert.	var.alert(datos);
confirm	Abre ventanas confirm.	var.confirm(datos);
prompt	Abre ventanas prompt.	var.prompt(datos,"valor inicial");
setInterval	Ejecuta una función cada cierto tiempo sin parar	v = setInterval(función, milisegundos);
clearInterval	Detiene la ejecución de setInterval	clearInterval(v);
setTimeout	Ejecuta una vez una función tras una espera	setTimeout(función, milisegundos);
La variable solo es necesaria cuando sea una ventana distinta a la del navegador.		

- El método **open** acepta como atributos una serie de **opción=valor,opción=valor,...**
 - **fullscreen**=yes o no
 - **height**=medida en pixels
 - **width**=medida en pixels
 - **top**=posición desde arriba en pixels
 - **left**=posición desde la izquierda en pixels

OBJETO WINDOW

PROPIEDAD	DESCRIPCIÓN
document	Devuelve la referencia al objeto Documento
history	Devuelve la referencia al objeto Historial
navigator	Devuelve la referencia al objeto Navegador
screen	Devuelve la referencia al objeto Pantalla
console	Devuelve la referencia al objeto Consola
innerHeight	Devuelve el alto del área de contenido de la pantalla incluyendo las barras de desplazamiento
innerWidth	Devuelve el ancho del área de contenido de la pantalla incluyendo las barras de desplazamiento
name	Devuelve o cambia el nombre de la ventana
screenX	Devuelve la posición horizontal de la ventana respecto a la pantalla
screenY	Devuelve la posición vertical de la ventana respecto a la pantalla
opener	Devuelve una referencia a la ventana padre
closed	Devuelve verdadero o falso si la ventana se ha cerrado

OBJETO DOCUMENT

- Representa la página web que se está visualizando y permite acceder a los elementos contenidos en ella

MÉTODO	DESCRIPCIÓN
getElementById(id)	Devuelve el elemento cuyo id se pasa al método
getElementsByName(nombre)	Devuelve una lista de los elementos con ese nombre
getElementsByTagName(etiqueta)	Devuelve una lista de los elementos con esa etiqueta
getElementsByClassName(clase)	Devuelve una lista de los elementos con esa clase CSS aplicada
querySelector(selectorCSS) querySelectorAll(selectorCSS)	Devuelve un elemento, o una lista de elementos, que coinciden con el selector CSS pasado como parámetro
addEventListener(evento, función)	Asigna un gestor para un evento. El nombre del evento irá sin la palabra on . La función irá sin paréntesis
removeEventListener(evento, func)	Elimina un gestor para un evento. El nombre del evento irá sin la palabra on . La función irá sin paréntesis

OBJETO DOCUMENT

PROPIEDAD	DESCRIPCIÓN
anchors	Devuelve una lista de todos los elementos <a> del documento
cookie	Devuelve una lista de todas las cookies del documento
characterSet	Devuelve la codificación del documento
forms	Devuelve una lista de todos los elementos <form> del documento
images	Devuelve una lista de todos los elementos del documento
title	Devuelve el título del documento

OBJETO CONSOLE

- Permite acceder a la consola de JavaScript, donde se pueden realizar impresiones para control y depuración

MÉTODO	DESCRIPCIÓN
log(mensaje)	Imprime un mensaje en la consola
info(mensaje)	Imprime un mensaje informativo en la consola
warn(mensaje)	Imprime un mensaje de aviso en la consola
error(mensaje)	Imprime un mensaje de error en la consola
group(nombreGrupo)	Crea un grupo para agrupar mensajes. La agrupación se representa mediante la indentación
groupEnd()	Finaliza un grupo creado con group()

OBJETO HISTORY

- Representa el historial de la sesión actual del navegador

MÉTODO	DESCRIPCIÓN
back()	Carga la página anterior
forward()	Carga la página siguiente
go(número)	Carga la página indicada por el número. Si es positivo va hacia adelante y si es negativo hacia atrás
ATRIBUTO	DESCRIPCIÓN
length	Devuelve el número de páginas del historial

OBJETO NAVIGATOR

- Representa a la aplicación que se está utilizando para visualizar la página web actual

PROPIEDAD	DESCRIPCIÓN
cookieEnabled	Indica si las cookies están habilitadas o no en el navegador
geolocation	Devuelve un objeto Geolocation que permite conocer la localización del usuario (si este lo aprueba)
language	Texto que representa el idioma preferido del usuario

OBJETO SCREEN

- Representa a la pantalla del dispositivo

PROPIEDAD	DESCRIPCIÓN
availHeight	Devuelve la altura de la pantalla (sin incluir la barra de tareas)
availWidth	Devuelve la anchura de la pantalla (sin incluir la barra de tareas)
height	Devuelve la altura total de la pantalla
width	Devuelve la anchura total de la pantalla
colorDepth	Devuelve la cantidad de colores (en bits) utilizada
pixelDepth	Devuelve la resolución de color (en bits por pixel)

GESTIÓN DE ERRORES

- En determinadas circunstancias, es posible que se produzcan errores al ejecutar un código Javascript, que se van a producir independientemente de las capacidades del programador
- En esos casos, se puede utilizar la siguiente sentencia para controlar esos errores

```
try {  
    //Instrucciones que pueden causar un error  
} catch (error) {  
    //Instrucciones para gestionar el error  
} finally {  
    //Código que se ejecuta siempre, haya o no error  
}
```

- El objeto **error** posee dos propiedades
 - **name**: Nombre del error que se produce
 - **message**: Descripción del error que se produce
 - Aunque cada navegador posee sus propias propiedades, estas son comunes

JSON

- Aunque XML es el estándar para el intercambio de información en Internet, cuando se necesita acceder a información estructurada en Javascript existe una mejor opción, que es JavaScript Object Notation
- En JSON se pueden representar 3 tipos de valores
 - Valores sencillos
 - Números enteros o decimales, cadenas de texto (string), valores lógicos (boolean) y nulos (null)
 - Objetos
 - Un conjunto de pares propiedad-valor en el que cada valor puede ser un elemento sencillo o compuesto
 - Arrays
 - Una lista ordenada de valores sencillo o compuestos accesibles por posición

JSON

- Valores sencillos
 - Simplemente se escribe el valor, entre comillas dobles si es texto

12 4.5 "hola" true
- Objetos
 - Se representan entre llaves, como una lista de pares propiedad (entre comillas dobles) : valor separados por comas

{ "nombre": "Juan", "edad": 25 }
- Arrays
 - Se representan como una lista separada por comas, entre corchetes

["lunes", "martes", 25 , true]
- Se pueden combinar estas tres representaciones como se quiera, creando arrays de objetos o objetos con propiedades de tipo array

JSON

- Para la manipulación de JSON existen dos métodos
 - **JSON.stringify(elemento)**
 - Convierte el elemento pasado como parámetro a texto JSON, devolviéndolo
 - Se puede incluir un método **toJSON** en el elemento para personalizar el comportamiento de este método
 - **JSON.parse(textoJSON)**
 - Convierte un texto JSON en el elemento correspondiente



METODOS Y FUNCIONES

METODOS DE TEXTO

- **length**
 - Es una propiedad y devuelve el número de caracteres del texto
- **indexOf (textoBuscado)**
 - Devuelve la primera posición de la primera ocurrencia del **textoBuscado**
 - En Javascript el texto empieza en la posición 0
- **lastIndexOf (textoBuscado)**
 - Devuelve la primera posición de la última ocurrencia del **textoBuscado**
- **slice (posiciónInicial, posiciónFinal)**
 - Devuelve el trozo de texto que se encuentra entre la **posiciónInicial** y la **posiciónFinal-1**
 - Si los valores de las posiciones son negativos empieza a contar por el final
- **substring (posiciónInicial, posiciónFinal)**
 - Igual que **slice**, pero no acepta valores negativos
- **substr (posiciónInicial, tamaño)**
 - Devuelve tantos caracteres como indica **tamaño** empezando en **posiciónInicial**

METODOS DE TEXTO

- **replace (textoOriginal, textoNuevo)**
 - Sustituye la primera aparición de **textoOriginal** por **textoNuevo**, devolviendo un nuevo texto. Con la expresión /g se puede hacer global p.ej: (/da/g, “de”)
- **toUpperCase()**
 - Devuelve el texto convertido todo a mayúsculas
- **toLowerCase()**
 - Devuelve el texto convertido todo a minúsculas
- **trim()**
 - Devuelve el texto eliminando todos los espacios sobrantes al principio y al final
- **charAt (posición)**
 - Devuelve el carácter que está en la **posición** indicada
 - La primera posición es 0
- **split (carácter)**
 - Devuelve un array resultado de separar el texto utilizando como separador el **carácter** pasado

METODOS DE TEXTO

Añadidos en 2017

- **padStart(tamañoTotal, stringRelleno)**
 - Completa el string añadiéndole por el principio el **stringRelleno** tantas veces como sea necesario hasta completar el **tamañoTotal** de caracteres
- **padEnd(tamañoTotal, stringRelleno)**
 - Completa el string añadiéndole por el final el **stringRelleno** tantas veces como sea necesario hasta completar el **tamañoTotal** de caracteres

FUNCIONES MATEMÁTICAS

- **Math.pow(base, exponente)**
 - Realiza la operación de potencia, elevando la base al exponente
- **Math.sqrt(número)**
 - Calcula la raíz cuadrada del número
- **Math.abs(número)**
 - Calcula el valor absoluto del número
- **Math.round(número)**
 - Redondea el número al entero más cercano
- **Math.ceil(número)**
 - Redondea el número al entero superior más cercano

FUNCIONES MATEMÁTICAS

- **Math.floor(numero)**
 - Redondea el número al entero inferior más cercano
- **Math.max(número, número, número...)**
 - Devuelve el mayor de los números pasados
- **Math.min(número, número, número....)**
 - Devuelve el menor de los números pasados
- **Math.random()**
 - Devuelve un número aleatorio entre 0 y 1 (no incluido)
- **Math.PI**
 - Devuelve el valor del número PI

METODOS DE FECHA Y HORA

- **getFullYear()**
 - Devuelve el año con 4 dígitos
- **getMonth()**
 - Devuelve el mes como un número de 0 a 11
- **getDate()**
 - Devuelve el día como un número de 1 a 31
- **getHours()**
 - Devuelve la hora como un número de 0 a 23
- **getMinutes()**
 - Devuelve los minutos como un número de 0 a 59
- **getSeconds()**
 - Devuelve los segundos como un número de 0 a 59
- **getMilliseconds()**
 - Devuelve los milisegundos como un número de 0 a 999
- **getTime()**
 - Devuelve los milisegundos que pasan desde el 1 de enero de 1970
- **getDay()**
 - Devuelve el día de la semana como un número de 0 (domingo) a 6 (sábado)

Los mismos métodos empezando por **set** se utilizan para modificar una fecha

METODOS DE FECHA Y HORA

- **toString()**
 - Pasa la fecha a un string en formato largo
 - *Wed Mar 09 2022 12:46:50 GMT+0100 (hora estándar de Europa central)*
- **toUTCString()**
 - Pasa la fecha a un string con formato UTC
 - *Wed, 09 Mar 2022 11:48:06 GMT*
- **toDateString()**
 - Pasa la fecha a un string más legible
 - *Wed Mar 09 2022*
- **toISOString()**
 - Pasa la fecha a un string en formato ISO
 - *2022-03-09T11:49:18.378Z*

METODOS DE ARRAYS

- **toString()**
 - Convierte un array en string
- **join(texto)**
 - Convierte un array en string, poniendo como separador el **texto**
- **pop()**
 - Devuelve y elimina el último elemento del array
- **push(elemento)**
 - Añade un nuevo **elemento** al final del array y devuelve la nueva longitud
- **shift()**
 - Devuelve y elimina el primer elemento del array
- **unshift(elemento)**
 - Añade un nuevo **elemento** al principio del array y devuelve la nueva longitud
- **concat(array1,array2,...)**
 - Crear un nuevo array al unir el array original con los pasados como parámetros

METODOS DE ARRAYS

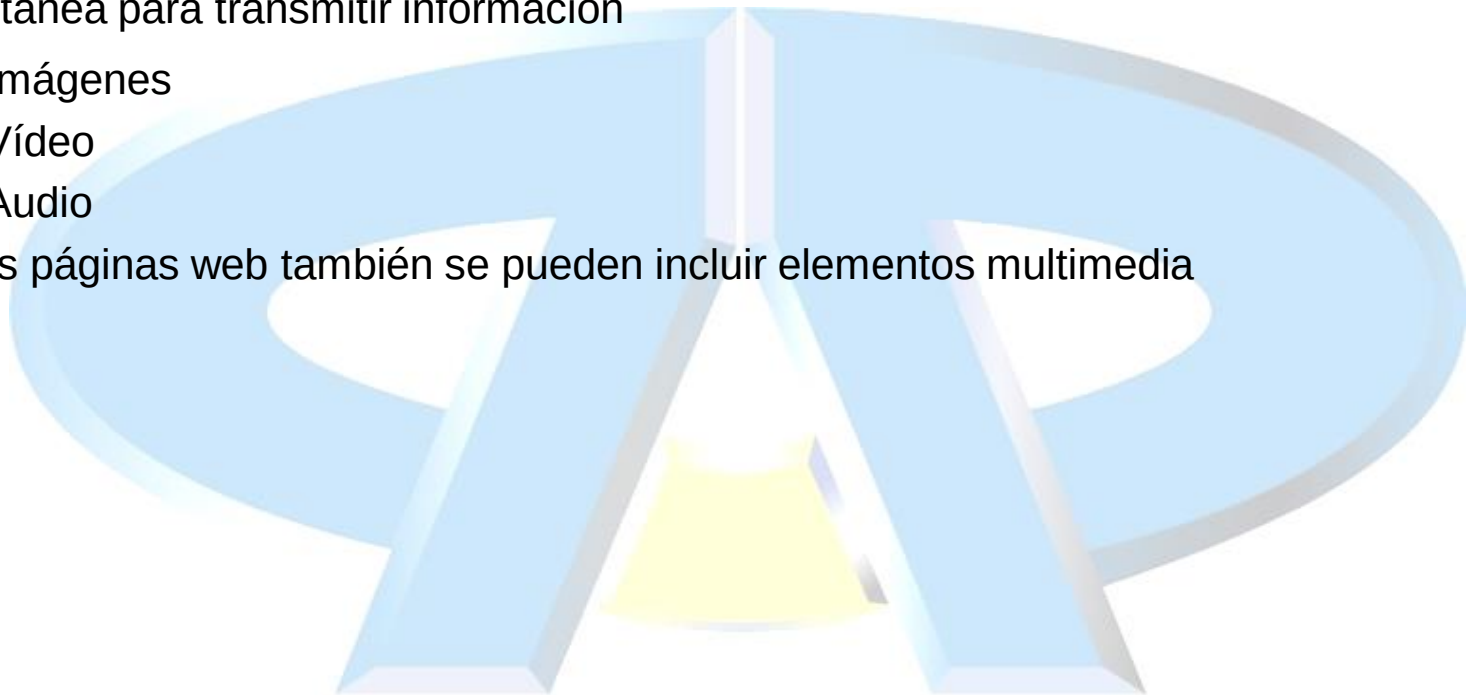
- **splice(posicion, numElementos, elemento1, elemento2,...)**
 - Devuelve un nuevo array con los **numElementos** elementos eliminados al insertar **elemento1, elemento2,...** a partir de la **posición**
 - Si no se le pasan los elementos como parámetros se puede utilizar para eliminar elementos sin dejar huecos en el array
- **slice(posicionInicial, posicionFinal)**
 - Extrae los elementos del array entre **posicionInicial** y **posicionFinal** (no incluido) sin eliminarlos del array
 - Si solo se le pasa la **posicionInicial**, extrae hasta el final del array
- **sort()**
 - Ordena un array ascendentemente según el alfabeto
 - Falla para ordenar números
- **reverse()**
 - Invierte el orden actual de los elementos de un array



CONTENIDOS MULTIMEDIA

TIPOS DE ELEMENTOS

- El contenido multimedia se refiere a todo el contenido que utiliza varios medios de forma simultánea para transmitir información
 - Imágenes
 - Vídeo
 - Audio
- En las páginas web también se pueden incluir elementos multimedia



AUDIO

- En el estándar de HTML5 solamente se soportan 3 tipos de formatos de audio:
 - MP3
 - WAV
 - OGG
- Para incluir audio en una página web se utiliza el elemento **<audio>**, que contendrá tantos elementos **<source>** como opciones alternativas en distintos formatos se quieran utilizar
 - También se incluirá dentro de **<audio>** un texto para el caso en que el navegador no soporte este elemento
- La etiqueta **<audio>** posee los siguientes atributos:
 - **autoplay**: El sonido empezará de forma automática (Actualmente no permitido hasta que el usuario interactúa con la página)
 - **controls**: Se mostrarán los controles del sonido (play, pause,...)
 - **loop**: El sonido se repetirá ininterrumpidamente
 - **src=url**: Se le pasa la URL del fichero de audio. Es preferible usar **<source>**

AUDIO

- La etiqueta **<source>** se utiliza para indicar la ubicación de los ficheros de audio y puede aparecer varias veces por si se quieren ofrecer formatos alternativos.
- El navegador elegirá uno de ellos
- Los atributos de la etiqueta **<source>** son:
 - **scr=url**: La dirección del fichero de audio
 - **media=condición**: Condiciona el sonido a un criterio del dispositivo
 - **type=tipo MIME**: Indica el tipo de formato del fichero de audio

VIDEO

- En el estándar de HTML5 solamente se soportan 3 tipos de formatos de video:
 - MP4
 - WEBM
 - OGG
- Para incluir un video en una página, se utiliza el elemento **<video></video>**, en el que, al igual que pasa con **<audio>**, se incluirán tantos elementos **<source>** como formatos alternativos se quieran ofrecer.
 - También se incluirá un texto para dar soporte al caso en el que el navegador no soporte este elemento
- La etiqueta **<video>** posee los siguientes atributos:
 - **autoplay, controls, loop**: Funcionan igual que con **<audio>**
 - **autoplay** se puede utilizar, si el video está inicialmente **muted**
 - **height=pixels, width=pixels**: Especifican el tamaño del video.
 - **poster=url**: Permite especificar una imagen para mostrar mientras se carga el video
 - **src=url**: Indica la URL del fichero de video que se quiere reproducir
- La etiqueta **<source>** es igual que para **<audio>**